



Birla Institute of Technology and Science Pilani, Pilani Campus

phirse lite lo

Siddhant Attavar, Arnav Gupta, Subham Pan

Inter Collegiate Programming Contest

2025-2026

Contest (1)

template.cpp

22 lines

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define int long long
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) ((int)((x).size()))
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

void solve() {

}

signed main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int t = 1;
    cin >> t;
    while (t--) solve();
}
```

.bashrc

2 lines

```
alias c='g++ -std=c++20 -fsanitize=undefined,address'
run() {c $1 && ./a.out < in.txt > out.txt && diff -w out.
    txt ans.txt}
```

hash.sh

1 lines

```
cpp -dD -P -fpreprocessed | tr -d '[:space:]' | md5sum |
    cut -c-6
```

Mathematics (2)

2.1 Sums

$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.2 Series

2.3 Probability theory

$$\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$$

2.3.1 Binomial Coefficients

$$\sum_{m=0}^n C(m, k) = C(n+1, k+1)$$

$$\sum_{k=0}^m C(n+k, k) = C(n+m+1, m)$$

$$1C(n, 1) + 2C(n, 2) + \dots + nC(n, n) = n2^{n-1}$$

$$C(n, 0) + C(n-1, 1) + \dots + C(n-k, k) + \dots + C(0, n) = F_{n+1}$$

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_type. To get a multiset, change less to less_equal

Time: $\mathcal{O}(\log N)$ <ext/pb_ds/assoc_container.hpp>, <ext/pb_ds/tree_policy.hpp>

455886, 14 lines

using namespace __gnu_pbds;**template<class** T**using** Tree = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

```

void example() {
    Tree<int> t, t2; t.insert(8);
    auto it = t.insert(10).first; t.join(t2);
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
}

```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).d77092, 5 lines**#include** <bits/extc++.h>

```

struct chash {
const uint64_t C=11(4e18*acos(0))|71;
11 operator()(11 x) const { return __builtin_bswap64(x*C)
    ; };
__gnu_pbds::gp_hash_table<11,int,chash> h({},{},{},{},{
    1<<16});

```

SegmentTree.h

Description: 1D point update and range query where cmb is any associative operation. $\text{seg}[1] == \text{query}(0, N-1)$.**Time:** $\mathcal{O}(\log N)$ e03cc5, 20 lines

```

template<typename T> struct SegTree { // cmb(ID,b) = b
    T ID; T (*cmb)(T a, T b);
    int n; vector<T> seg;
    SegTree(int _n, T id, T _cmb(T, T)) {

```

```

        ID = id; cmb = _cmb;
        for (n = 1; n < _n; ) n *= 2;
        seg.assign(2*n, ID);
    }
    void pull(int p) { seg[p] = cmb(seg[2*p], seg[2*p+1]); }
    void upd(int p, T val) { // set val at position p
        seg[p += n] = val; for (p /= 2; p; p /= 2) pull(p); }
    T query(int l, int r) { // zero-indexed, inclusive
        T ra = ID, rb = ID;
        for (l += n, r += n+1; l < r; l /= 2, r /= 2) {
            if (l&1) ra = cmb(ra, seg[l++]);
            if (r&1) rb = cmb(seg[--r], rb);
        }
        return cmb(ra, rb);
    }
};

```

SegmentTreeRecursive.h

Description: 1D point update and range query where cmb is any associative operation. $\text{seg}[1] == \text{query}(0, N-1)$.**Time:** $\mathcal{O}(\log N)$ 2a972c, 31 lines

```

template<typename T> struct SegTreeRecursive {
    vector<T> seg; T id; T (*cmb)(T, T); int SZ;
    SegTreeRecursive(int _n, T _id, T _cmb(T, T)):SZ(_n),
        id(_id), cmb(_cmb){seg.assign(4*SZ+1, id);}
    void build() {for (int i=SZ-1; i >= 0; i--) seg[i]=
        cmb(seg[2*i], seg[2*i+1]);}
    T query(int l, int r) {return query(l, r, 0, SZ-1, 1);}
    T query(int l, int r, int s, int e, int c) {
        if (l > e || r < s) return id;
        if (l <= s && r >= e) return seg[c];
        int mid = (s + e) / 2;
        return cmb(query(l, r, s, mid, 2*c),
            query(l, r, mid + 1, e, 2*c+1));
    }
    void upd(int i, T x) {upd(i, x, 0, SZ-1, 1);}
    void upd(int i, T x, int s, int e, int c) {

```

```

    if (s > i || e < i) return;
    if (s == e) {seg[c] = x; return;}
    int mid = (s + e) / 2;
    upd(i, x, s, mid, 2*c);
    upd(i, x, mid + 1, e, 2*c+1);
    seg[c] = cmb(seg[2*c], seg[2*c+1]);
}
T kth_one_end(int k) {return kth_one_end(k, 0, SZ-1, 1);
}
T kth_one_end(int k, int s, int e, int c) {
    if(seg[c] < k) { return -1; }
    if(s == e) { return s; }
    int mid = (s+e)/2;
    if(seg[2*c] >= k) { // descend into right subtree
        return kth_one_end(k, s, mid, 2*c);}
    else {return kth_one_end(k - seg[2*c], mid+1, e,
        2*c+1);}
}
};

```

LazySegmentTree.h

Description: 1D range increment and sum query.

Usage: Node* tr = new Node(v, 0, sz(v));

Time: $\mathcal{O}(\log N)$.

71909a, 28 lines

```

template<class T, class U, int SZ> struct LazySeg {
    // static_assert(pct(SZ) == 1); // SZ must be power of
    2
    const T ID1; T (*cmb)(T a, T b);
    vector<T> seg; vector<U> lazy;
    void (*push)(int,int,int,vector<T>&,vector<U>&) =
        [&](int ind,int L,int R,vector<T>&seg,vector<U>&lazy)
        {
            seg[ind] += (R-L+1)*lazy[ind]; // dependent on
            operation
            if (L != R) {lazy[2*ind] += lazy[ind]; lazy[2*ind+1]
                += lazy[ind];}
            lazy[ind] = 0;

```

```

}; // recalc values for current node
LazySeg(T id1, U id2, T _cmb(T, T), void _push(int,int,
    int,vector<T>&,vector<U>&)):
    ID1(id1),cmb(_cmb),push(_push),seg(2*SZ,id1),lazy(2*
        SZ,id2){}
void pull(int ind){seg[ind]=cmb(seg[2*ind],seg[2*ind
    +1]);}
void build() { for(int i=SZ-1;i>=1;i--) pull(i); }
void upd(int lo,int hi,U inc,int ind=1,int L=0, int R=
    SZ-1) {
    push(ind,L,R,seg,lazy); if (hi < L || R < lo) return;
    if (lo <= L && R <= hi) {
        lazy[ind] = inc; push(ind,L,R,seg,lazy); return; }
    int M = (L+R)/2; upd(lo,hi,inc,2*ind,L,M);
    upd(lo,hi,inc,2*ind+1,M+1,R); pull(ind);
}
T query(int lo, int hi, int ind=1, int L=0, int R=SZ-1)
{
    push(ind,L,R,seg,lazy); if (lo > R || L > hi) return
        ID1;
    if (lo <= L && R <= hi) return seg[ind];
    int M = (L+R)/2; return cmb(query(lo,hi,2*ind,L,M),
        query(lo,hi,2*ind+1,M+1,R));
}
};

```

SparseSegmentTree.h

Description: Sparse/Persistent Segment Tree. Possible MLE/TLE issues. Provide approx q = num of queries + updates to improve time per operation

Time: $\mathcal{O}(\log N)$

0530fa, 31 lines

```

template<typename T, int SZ> struct SparseSegmentTree {
    vector<pair<int, int>> tree;
    vector<T> seg; T id; int n;
    T (*cmb)(T, T);
    SparseSegmentTree(T _id, T _cmb(T, T), int q = 0):id(
        _id,
        cmb(_cmb),n(1),tree({{-1, -1}}),seg({{id}}) {

```

```

    int k = 2 * q * __lg(SZ);
    tree.reserve(k); seg.reserve(k);
}
void add(int c, int l, int r) {
    if (tree[c].first != -1 or l == r) return;
    tree[c] = {n, n + 1}; n += 2;
    tree.push_back({-1, -1}); tree.push_back({-1, -1});
    seg.push_back(id); seg.push_back(id);
}
T query(int l, int r, int s=0, int e=SZ-1, int c=0) {
    if (l > e || r < s) return id;
    if (l <= s && r >= e) return seg[c];
    int mid = (s + e) / 2; add(c, s, e);
    return cmb(query(l, r, s, mid, tree[c].first),
               query(l, r, mid + 1, e, tree[c].second));
}
void upd(int i, T x, int s=0, int e=SZ-1, int c=0) {
    if (s > i || e < i) return;
    if (s == e) {seg[c] = x; return;}
    int mid = (s + e) / 2; add(c, s, e);
    upd(i, x, s, mid, tree[c].first);
    upd(i, x, mid + 1, e, tree[c].second);
    seg[c] = cmb(seg[tree[c].first], seg[tree[c].second]);
}
};

```

PersistentSegTree.h

Description: Persistent SegTree

Time: $\mathcal{O}(\log N)$

2a07cd, 45 lines

```

struct PersistentSegTree
{ // Returns the sum of K largest elements in [l, r]
    map<int, pii> cp;
    // add log2(Q) is storage needed after some update
    query
    int L[LN*N], R[LN*N], S[LN*N]; long long KS[LN*N];
    int SZ = 1, roots[N], MX;

```

```

PersistentSegTree(int v[], int n) : MX(n) {
    int tl = 0, tr = n, idx[n]; pii sort_v[n];
    rep(i, 0, n) sort_v[i] = {v[i], i};
    sort(sort_v, sort_v+n, greater<pii>());
    rep(i, 0, n) cp[i] = sort_v[i], idx[sort_v[i].second] = i;
    roots[0] = build(tl, tr);
    rep(i, 0, n) roots[i+1] = update(roots[i], tl, tr, idx[i]);
}
int ver(int s, int ks, int l, int r) {
    S[SZ] = s; KS[SZ] = ks; L[SZ] = l; R[SZ] = r; return SZ++;
}
int ver(int l, int r) {
    S[SZ] = S[l] + S[r]; KS[SZ] = KS[l] + KS[r];
    L[SZ] = l; R[SZ] = r; return SZ++;
}
int build(int tl, int tr) {
    if (tl == tr) return ver(0, 0, -1, -1);
    int tm = (tl + tr) / 2;
    return ver(build(tl, tm), build(tm + 1, tr));
}
int update(int idx, int tl, int tr, int pos) {
    if (tl == tr)
        return ver(S[idx] + 1, KS[idx] + cp[pos].first,
                   -1, -1);
    int tm = (tl + tr) / 2;
    if (pos <= tm)
        return ver(update(L[idx], tl, tm, pos), R[idx]);
    return ver(L[idx], update(R[idx], tm + 1, tr, pos));
}
long long kth(int vl, int vr, int tl, int tr, int k)
{
    if (tl == tr) return KS[vr] - KS[vl];
    int tm = (tl + tr) / 2, lc = S[L[vr]] - S[L[vl]];
    if (lc >= k) return kth(L[vl], L[vr], tl, tm, k);
    return KS[L[vr]] - KS[L[vl]] + kth(R[vl],

```

```

        R[vr], tm+1, tr, k-lc);
    }
    long long kth(int l, int r, int k) {
        return kth(roots[l], roots[r], 0, MX, k);
    }
};

```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip `st, time()` and `rollback()`.

Usage: `int t = uf.time(); ...; uf.rollback(t);`

Time: $\mathcal{O}(\log N)$

de4ad0, 21 lines

```

struct RollbackUF {
    vi e; vector<pii> st;
    RollbackUF(int n) : e(n, -1) {}
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    int time() { return sz(st); }
    void rollback(int t) {
        for (int i = time(); i --> t;)
            e[st[i].first] = st[i].second;
        st.resize(t);
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        st.push_back({a, e[a]});
        st.push_back({b, e[b]});
        e[a] += e[b]; e[b] = a;
        return true;
    }
};

```

LineContainer.h

Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).

Time: $\mathcal{O}(\log N)$

8ec1c7, 30 lines

```

struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};

```

RMQ.h

Description: Range Minimum Queries on an array. Returns $\min(V[a], V[a + 1], \dots, V[b - 1])$ in constant time.

Usage: RMQ rmq(values);

rmq.query(inclusive, inclusive);

Time: $\mathcal{O}(|V| \log |V| + Q)$

f33baf, 16 lines

```
template<class T>
struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j, 0, sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) { b++; // (for right inclusive)
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};
```

Numerical (4)

4.1 Matrices

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.

Time: $\mathcal{O}(n^2m)$

44c9ab, 38 lines

```
typedef vector<double> vd;
const double eps = 1e-12;
```

```
int solveLinear(vector<vd>& A, vd& b, vd& x) {
    int n = sz(A), m = sz(x), rank = 0, br, bc;
    if (n) assert(sz(A[0]) == m);
    vi col(m); iota(all(col), 0);

    rep(i, 0, n) {
        double v, bv = 0;
        rep(r, i, n) rep(c, i, m)
            if ((v = fabs(A[r][c])) > bv)
                br = r, bc = c, bv = v;
        if (bv <= eps) {
            rep(j, i, n) if (fabs(b[j]) > eps) return -1;
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j, 0, n) swap(A[j][i], A[j][bc]);
        bv = 1/A[i][i];
        rep(j, i+1, n) {
            double fac = A[j][i] * bv;
            b[j] -= fac * b[i];
            rep(k, i+1, m) A[j][k] -= fac * A[i][k];
        }
        rank++;
    }

    x.assign(m, 0);
    for (int i = rank; i--;) {
        b[i] /= A[i][i];
        x[col[i]] = b[i];
        rep(j, 0, i) b[j] -= A[j][i] * b[i];
    }
    return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

"SolveLinear.h" 08e495, 7 lines

```
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
    rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
    x[col[i]] = b[i] / A[i][i];
fail:; }
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .

Time: $\mathcal{O}(n^2m)$

typedef bitset<1000> bs;

```
int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
    int n = sz(A), rank = 0, br;
    assert(m <= sz(x));
    vi col(m); iota(all(col), 0);
    rep(i,0,n) {
        for (br=i; br<n; ++br) if (A[br].any()) break;
        if (br == n) {
            rep(j,i,n) if (b[j]) return -1;
            break;
        }
        int bc = (int)A[br]._Find_next(i-1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) if (A[j][i] != A[j][bc]) {
            A[j].flip(i); A[j].flip(bc);
        }
        rep(j,i+1,n) if (A[j][i]) {
            b[j] ^= b[i];
        }
    }
}
```

```
    A[j] ^= A[i];
}
rank++;
}

x = bs();
for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
}
return rank; // (multiple solutions if rank < m)
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.

If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.

Time: $\mathcal{O}(N)$

8f9fa8, 26 lines


```

typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>&
    super,
    const vector<T>& sub, vector<T> b) {
    int n = sz(b); vi tr(n);
    rep(i,0,n-1) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i]
            == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[++i] = 1;
        } else {
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
    for (int i = n; i--;) {
        if (tr[i]) {
            swap(b[i], b[i-1]);
            diag[i-1] = diag[i];
            b[i] /= super[i-1];
        } else {
            b[i] /= diag[i];
            if (i) b[i-1] -= b[i]*super[i-1];
        }
    }
    return b;
}

```

4.2 Fourier transforms

FastFourierTransform.h

Description: $\text{fft}(a)$ computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: $\text{conv}(a, b) = c$, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n , reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

00ced6, 35 lines

```

typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
    }
vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
    rep(i,0,sz(b)) in[i].imag(b[i]);
}

```

```

fft(in);
for (C& x : in) x *= x;
rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
fft(out);
rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
return res;
}

```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h" cad46c, 22 lines

```

typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=(int)(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = (ll)(real(outl[i])+.5), cv = (ll)(imag(outs[i])+.5);

```

```

        ll bv = (ll)(imag(outl[i])+.5) + (ll)(real(outs[i])+.5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}

```

NumberTheoreticTransform.h

Description: $\text{ntt}(a)$ computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(\text{mod}-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a . For arbitrary modulo, see FFTMod. $\text{conv}(a, b) = c$, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply point-wise, divide by n, reverse(start+1, end), NTT back. Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h" ced03d, 35 lines

```

const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479
    << 21
// and 483 << 21 (same root). The last two are > 10^9.

```

```

typedef vector<ll> vl;
void ntt(vl &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow(root, mod >> s)};
        rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
            a[i + j + k] = ai - z + (z > ai ? mod : 0);

```

```

        ai += (ai + z >= mod ? z - mod : z);
    }
}
vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
        n = 1 << B;
    int inv = modpow(n, mod - 2);
    vl L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i, 0, n)
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}

```

Number theory (5)

5.1 Modular arithmetic

ModPow.h

b83e45, 8 lines

```

const ll mod = 1000000007; // faster if const

ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}

```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

bbbd8f, 11 lines

```

typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}

```

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.

Time: LIM=1e9 $\approx$ 1.5s

6b2912, 20 lines

```

const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)
        *1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
        rep(i, 0, min(S, R - L))
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
}

```

```

    return pr;
}

```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.

Time: 7 times the complexity of $a^b \bmod c$.

```

"ModMullL.h" 60dcd1, 12 lines
bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504,
               1795265022},
    s = __builtin_ctzll(n-1), d = n >> s;
    for (ull a : A) { // ^ count trailing zeroes
        ull p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}

```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).

Time: $\mathcal{O}(n^{1/4})$, less for numbers with small factors.

```

"ModMullL.h", "MillerRabin.h" d8d98d, 18 lines
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd =
            q;
        x = f(x), y = f(f(y));
    }
}

```

```

    return __gcd(prd, n);
}
vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}

```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod{b}$.

```

33ba8f, 5 lines
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}

```

CRT.h

Description: Chinese Remainder Theorem.

`crt(a, m, b, n)` computes x such that $x \equiv a \pmod{m}$, $x \equiv b \pmod{n}$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.

Time: $\log(n)$

```

"euclid.h" 04d93a, 7 lines
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}

```

5.3.1 Bézout's identity

For $a \neq 0$, $b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a, b)}, y - \frac{ka}{\gcd(a, b)} \right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: Euler's ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1-1)p_1^{k_1-1} \dots (p_r-1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$.

$\sum_{d|n} \phi(d) = n$, $\sum_{k=1, \gcd(k, n)=1}^n k = n\phi(n)/2, n > 1$

Euler's: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod{n}$.

Fermat's little: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod{p} \forall a$.

cf7d6d, 8 lines

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
    rep(i, 0, LIM) phi[i] = i & 1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2) if (phi[i] == i)
        for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

5.5 Estimates

$$\sum_{d|n} d = O(n \log \log n).$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.6 Mobius Function

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$

Combinatorial (6)

6.0.1 Labeled unrooted trees

on n vertices: n^{n-2}

on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$

with degrees d_i : $(n-2)! / ((d_1-1)! \dots (d_n-1)!)$

6.0.2 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, C_{n+1} = \frac{2(2n+1)}{n+2}C_n, C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

7.2 Network flow

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

4b9fe1, 79 lines

```
#include <bits/extc++.h>
```

```
const ll INF = numeric_limits<ll>::max() / 4;
```

```
struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
```

```
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;
```

```
MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}
```

```
void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(edge{ from, to, (int)sz(ed[to]), cap,
        cost, 0 });
    ed[to].push_back(edge{ to, from, (int)sz(ed[from])
        -1, 0, -cost, 0 });
}
```

```
void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0; ll di;
```

```
    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({ 0, s });
```

```
while (!q.empty()) {
    s = q.top().second; q.pop();
    seen[s] = 1; di = dist[s] + pi[s];
    for (edge& e : ed[s]) if (!seen[e.to]) {
        ll val = di - pi[e.to] + e.cost;
        if (e.cap - e.flow > 0 && val < dist[e.to]) {
            dist[e.to] = val;
            par[e.to] = &e;
            if (its[e.to] == q.end())
                its[e.to] = q.push({ -dist[e.to], e.to });
            else
                q.modify(its[e.to], { -dist[e.to], e.to });
        }
    }
}
```

```

    }
}
rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
}

pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
        ll fl = INF;
        for (edge* x = par[t]; x; x = par[x->from])
            fl = min(fl, x->cap - x->flow);

        totflow += fl;
        for (edge* x = par[t]; x; x = par[x->from]) {
            x->flow += fl;
            ed[x->to][x->rev].flow -= fl;
        }
    }
    rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e
        .flow;
    return {totflow, totcost/2};
}

// If some costs can be negative, call this before
// maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
        rep(i,0,N) if (pi[i] != INF)
            for (edge& e : ed[i]) if (e.cap)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}
};

```

Dinic.h

Description: Flow algorithm with complexity $O(VE \log U)$ where $U = \max |cap|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{VE})$ for bipartite matching.

d7f0f1, 42 lines

```

struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need
            flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L,0,31) do { // 'int L=30' maybe faster for
            random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];

```

```

    for (Edge e : adj[v])
        if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
    }
    while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
} while (lvl[t]);
return flow;
}
bool leftOfMinCut(int a) { return lvl[a] != 0; }
};

```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

7.3 Matching

hopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Usage: `vi btoa(m, -1); hopcroftKarp(g, btoa);`

Time: $\mathcal{O}(\sqrt{VE})$

f612e4, 42 lines

```

bool dfs(int a, int L, vector<vi>& g, vi& btoa, vi& A, vi
    & B) {
    if (A[a] != L) return 0;
    A[a] = -1;
    for (int b : g[a]) if (B[b] == L + 1) {
        B[b] = 0;
        if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A,
            B))
            return btoa[b] = a, 1;
    }
    return 0;
}

```

```

}
int hopcroftKarp(vector<vi>& g, vi& btoa) {
    int res = 0;
    vi A(g.size()), B(btoa.size()), cur, next;
    for (;;) {
        fill(all(A), 0);
        fill(all(B), 0);
        cur.clear();
        for (int a : btoa) if (a != -1) A[a] = -1;
        rep(a, 0, sz(g)) if (A[a] == 0) cur.push_back(a);
        for (int lay = 1;; lay++) {
            bool islast = 0;
            next.clear();
            for (int a : cur) for (int b : g[a]) {
                if (btoa[b] == -1) {
                    B[b] = lay;
                    islast = 1;
                }
                else if (btoa[b] != a && !B[b]) {
                    B[b] = lay;
                    next.push_back(btoa[b]);
                }
            }
            if (islast) break;
            if (next.empty()) return res;
            for (int a : next) A[a] = lay;
            cur.swap(next);
        }
        rep(a, 0, sz(g))
            res += dfs(a, 0, g, btoa, A, B);
    }
}

```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes $\text{cost}[N][M]$, where $\text{cost}[i][j] = \text{cost}$ for $L[i]$ to be matched with $R[j]$ and returns $(\min \text{cost}, \text{match})$, where $L[i]$ is matched with $R[\text{match}[i]]$. Negate costs for max cost. Requires $N \leq M$.

Time: $\mathcal{O}(N^2M)$

1e0fe9, 31 lines

```
pair<int, vi> hungarian(const vector<vi> &a) {
    if (a.empty()) return {0, {}};
    int n = sz(a) + 1, m = sz(a[0]) + 1;
    vi u(n), v(m), p(m), ans(n - 1);
    rep(i, 1, n) {
        p[0] = i;
        int j0 = 0; // add "dummy" worker 0
        vi dist(m, INT_MAX), pre(m, -1);
        vector<bool> done(m + 1);
        do { // dijkstra
            done[j0] = true;
            int i0 = p[j0], j1, delta = INT_MAX;
            rep(j, 1, m) if (!done[j]) {
                auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta) delta = dist[j], j1 = j;
            }
            rep(j, 0, m) {
                if (done[j]) u[p[j]] += delta, v[j] -= delta;
                else dist[j] -= delta;
            }
            j0 = j1;
        } while (p[j0]);
        while (j0) { // update alternating path
            int j1 = pre[j0];
            p[j0] = p[j1], j0 = j1;
        }
    }
    rep(j, 1, m) if (p[j]) ans[p[j] - 1] = j - 1;
    return {-v[0], ans}; // min cost
}
```

7.4 DFS algorithms

CutVerticesAndEdges.h

Description: Finding cut vertices and cut edges.

88a95c, 43 lines

```
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> tin, low;
int timer;
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0; // CUT VERTICE
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v]) // CUT EDGE
                IS_BRIDGE(v, to);
        }
    }
    if (p == -1 && children > 1) // CUT VERTICE
        IS_CUTPOINT(v);
}
void find_cutpoints() { // CUT VERTICE
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}
```

```

void find_bridges() { // CUT EDGE
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}

```

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage: `scc(graph, [&](vi& v) { ... })` visits all components

in reverse topological order. `comp[i]` holds the component

index of a node (a component only has edges to components with

lower index). `ncomps` will contain the number of components.

Time: $\mathcal{O}(E + V)$

76b5c9, 24 lines

```

vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
    int low = val[j] = ++Time, x; z.push_back(j);
    for (auto e : g[j]) if (comp[e] < 0)
        low = min(low, val[e] ? : dfs(e, g, f));

    if (low == val[j]) {
        do {
            x = z.back(); z.pop_back();
            comp[x] = ncomps;
            cont.push_back(x);
        } while (x != j);
        f(cont); cont.clear();
    }
}

```

```

        ncomps++;
    }
    return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
    int n = sz(g);
    val.assign(n, 0); comp.assign(n, -1);
    Time = ncomps = 0;
    rep(i, 0, n) if (comp[i] < 0) dfs(i, g, f);
}

```

2sat.h

Description: Calculates a valid assignment to boolean variables $a, b, c, \dots$ to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&\dots$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions ($\sim x$).

Usage: `TwoSat ts(number of boolean variables);`

`ts.either(0, ~3);` // Var 0 is true or var 3 is false

`ts.setValue(2);` // Var 2 is true

`ts.atMostOne({0, ~1, 2});` // ≤ 1 of vars 0, ~1 and 2 are true

`ts.solve();` // Returns true iff it is solvable

`ts.values[0..N-1]` holds the assigned values to the vars

Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

5f9706, 56 lines

```

struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n) {}

    int addVar() { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }
}

```

```

void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
}

void setValue(int x) { either(x, x); }

void atMostOne(const vi& li) { // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i, 2, sz(li)) {
        int next = addVar();
        either(cur, ~li[i]);
        either(cur, next);
        either(~li[i], next);
        cur = ~next;
    }
    either(cur, ~li[1]);
}

vi val, comp, z; int time = 0;
int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
        low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
        x = z.back(); z.pop_back();
        comp[x] = low;
        if (values[x>>1] == -1)
            values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
}

bool solve() {

```

```

        values.assign(N, -1);
        val.assign(2*N, 0); comp = val;
        rep(i, 0, 2*N) if (!comp[i]) dfs(i);
        rep(i, 0, N) if (comp[2*i] == comp[2*i+1]) return 0;
        return 1;
    }
};

```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.

Time: $\mathcal{O}(V + E)$

780b64, 15 lines

```

vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src
=0) {
    int n = sz(gr);
    vi D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = sz(gr[x])
        ;
        if (it == end){ ret.push_back(x); s.pop_back();
            continue; }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--, D[y]++;
            eu[e] = 1; s.push_back(y);
        }
    }
    for (int x : D) if (x < 0 || sz(ret) != nedges+1)
        return {};
    return {ret.rbegin(), ret.rend()};
}

```

7.5 Trees

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.

Time: $\mathcal{O}(N \log N + Q)$

```

"../data-structures/RMQ.h"
d9fc67, 21 lines

struct LCA {
    int T = 0;
    vi time, path, ret;
    RMQ<int> rmq;

    LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret
    )) {}
    void dfs(vector<vi>& C, int v, int par) {
        time[v] = T++;
        for (int y : C[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            dfs(C, y, v);
        }
    }

    int lca(int a, int b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b - 1)];
    }
    //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,
    b)];}
};

```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.

Time: $\mathcal{O}(|S| \log |S|)$

```

"LCA.h"
9775a0, 21 lines

typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
    static vi rev; rev.resize(sz(lca.time));
    vi li = subset, &T = lca.time;
    auto cmp = [&](int a, int b) { return T[a] < T[b]; };
    sort(all(li), cmp);
    int m = sz(li)-1;
    rep(i,0,m) {
        int a = li[i], b = li[i+1];
        li.push_back(lca.lca(a, b));
    }
    sort(all(li), cmp);
    li.erase(unique(all(li)), li.end());
    rep(i,0,sz(li)) rev[li[i]] = i;
    vpi ret = {pii(0, li[0])};
    rep(i,0,sz(li)-1) {
        int a = li[i], b = li[i+1];
        ret.emplace_back(rev[lca.lca(a, b)], b);
    }
    return ret;
}

```

Centroid.h

Description: Find centroid decomposition of a tree. Main function to be called is solve(root, root, [(ll v)). Process trees formed after decomposition at each step within process(node)

Time: $\mathcal{O}(N \log N)$

```

cb3bfe, 35 lines

struct Centroid { // 0 based indexing.
    vector<vi> centroid_tree, adj;
    vi par_ct, siz;
    vector<bool> vis;
    Centroid(vector<vector<ll>> _adj) : adj(_adj),
        cent_tree(sz(_adj)),
        siz(sz(_adj)), vis(sz(_adj)), par_ct(sz(_adj), -1) {}
    void find_size(ll v, ll par) {

```

```

    siz[v] = 1;
    for(auto x : adj[v]) {
        if(x == par || vis[x]) continue;
        find_size(x, v);
        siz[v] += siz[x];
    }
}
ll find_centroid(ll v, ll par, ll n_nodes) {
    for(auto x : adj[v]) {
        if(x == par || vis[x]) continue;
        if(siz[x] > n_nodes/2) return find_centroid(x, v,
            n_nodes);
    }
    return v;
}
void process(ll v) {} // optional
ll solve(ll v, ll prev) {
    find_size(v, prev);
    ll centroid = find_centroid(v, prev, siz[v]);
    vis[centroid] = true;
    par_ct[centroid] = prev;
    process(centroid);
    for(auto x : adj[centroid]) {
        if (vis[x]) continue;
        centroid_tree[centroid].push_back(solve(x, centroid
        ));
    }
    return centroid;
}
};

```

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 28 lines

```

template <class T> int sgn(T x) { return (x > 0) - (x <
    0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p
        .y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p
        .y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*
        this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // makes dist()
        =1
    P perp() const { return P(-y, x); } // rotates +90
        degrees
    P normal() const { return perp().unit(); }

```

```
// returns point rotated 'a' radians ccw around the
// origin
P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

lineDistance.h

Description: Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan.

```
"Point.h" f6bf6b, 4 lines
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
    return (double) (b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

Description: Returns the shortest distance between point p and the line segment from point s to e.

Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

```
"Point.h" 5c88f4, 6 lines
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
    if (s==e) return (p-s).dist();
    auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)
    ));
    return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h

Description:

If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned as a vector of length 1. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates.

```
"Point.h", "OnSegment.h" 9d57f2, 13 lines
template<class P> vector<P> segInter(P a, P b, P c, P d)
{
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```

lineIntersection.h

Description:

If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates.

```
"Point.h" a01f81, 8 lines
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

```
}

```

sideOf.h

Description: Returns where p is as seen from s towards e . $1/0/-1 \Leftrightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be $\text{Point}<T>$ where T is e.g. double or long long.

```
"Point.h" c3af81c, 9 lines
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    {
        auto a = (e-s).cross(p-s);
        double l = (e-s).dist()*eps;
        return (a > l) - (a < -l);
    }
}
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e . Use $(\text{segDist}(s,e,p) \leq \text{epsilon})$ instead when using $\text{Point}<\text{double}>$.

```
"Point.h" c597e8, 3 lines
template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

Description:

Apply the linear transformation (translation, rotation and scaling) which takes line p_0 - p_1 to line q_0 - q_1 to point r .

```
"Point.h" 03a306, 6 lines
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
    P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq))
    ;
}
```

```
return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.
    dist2());
}
```

LineProjectionReflection.h

Description: Projects point p onto line ab . Set $\text{refl}=\text{true}$ to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates.

```
"Point.h" b5562d, 5 lines
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
    P v = b - a;
    return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

8.2 Circles

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be $\text{Point}<\text{double}>$.

```
"Point.h" e0cfba, 9 lines
template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
    P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
    double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
    if (h2 < 0) return {};
    if (h2 == 0) return {p};
    P h = ab.unit() * sqrt(h2);
    return {p - h, p + h};
}
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`

`bool in = inPolygon(v, P{3, 3}, false);`

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h"

2bf504, 11 lines

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = sz(p);
    rep(i, 0, n) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !strict;
        //or: if (segDist(p[i], q, a) <= eps) return !strict;
        cnt ^= ((a.y < p[i].y) - (a.y < q.y)) * a.cross(p[i], q)
              > 0;
    }
    return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h"

f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    rep(i, 0, sz(v)-1) a += v[i].cross(v[i+1]);
    return a;
}
```

ConvexHull.h

Description:

Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

"Point.h"

310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
```

```
    if (sz(pts) <= 1) return pts;
    sort(all(pts));
    vector<P> h(sz(pts)+1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0)
                t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h"

71446b, 14 lines

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true)
{
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
    ;
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)
        <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```


8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

```
"Point.h" ac41a6, 17 lines

typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d{1 + (ll)sqrt(ret.first), 0};
        while (v[j].y <= p.y - d.x) S.erase(v[j++]);
        auto lo = S.lower_bound(p - d), hi = S.upper_bound(p
            + d);
        for (; lo != hi; ++lo)
            ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
        S.insert(p);
    }
    return ret.second;
}
```

Strings (9)

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

```
d4375c, 16 lines

vi pi(const string& s) {
    vi p(sz(s));
    rep(i, 1, sz(s)) {
        int g = p[i-1];
```

```
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}

vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i, sz(p)-sz(s), sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

```
ee09e2, 12 lines

vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i, 1, sz(S)) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).

Time: $\mathcal{O}(N)$

```
e7ad79, 13 lines

array<vi, 2> manacher(const string& s) {
    int n = sz(s);
```

```

array<vi,2> p = {vi(n+1), vi(n)};
rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
        p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
}
return p;
}

```

SuffixArray.h

Description: Builds suffix array for a string. $sa[i]$ is the starting index of the suffix which is i 'th in the sorted suffix array. The returned vector is of size $n+1$, and $sa[0] = n$. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: $lcp[i] = lcp(sa[i], sa[i-1])$, $lcp[0] = 0$. The input string must not contain any nul chars.

Time: $\mathcal{O}(n \log n)$

635552, 22 lines

```

struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string s, int lim=256) { // or vector<int>
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim
            = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 :
                    p++;
        }
    }
}

```

```

}
for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
    for (k && k--, j = sa[x[i] - 1];
        s[i + k] == s[j + k]; k++);
}
};

```

Various (10)

10.1 Misc. algorithms

RNG.h

Description: Fast random number generator.

Usage: `rng(); shuffle(all(v), rng);`

c8a385, 1 lines

```

mt19937 rng(chrono::steady_clock::now().time_since_epoch
    ().count());

```

10.2 Optimization tricks

10.2.1 Bit hacks

- $x \& -x$ is the least bit in x .
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of m (except m itself).
- $c = x \& -x$, $r = x + c$; $((r \ll x) >> 2) / c$ | r is the next number after x with the same number of bits set.
- `rep(b,0,K) rep(i,0,(1 << K))`
`if (i & 1 << b) D[i] += D[i ^ (1 << b)];` computes all sums of subsets.